CSE508 Network Security

Information Flow and the CBC Padding Oracle Attack

Ka Lok Wu
April 28, 2026

Acknowledgement

Some of the contents in this slide deck are adapted from:

- Prof. Sze Yiu Chau (Department of Information Engineering, the Chinese University of Hong Kong)
- Serge Vaudenay. Security flaws induced by CBC padding—applications to SSL, IPSEC, WTLS... (EUROCRYPT 2002)
- Brice Canvel, Alain Hiltgen, Serge Vaudenay, and Martin Vuagnoux. Password interception in a SSL/TLS channel. (CRYPTO 2003)
- Nadhem J. AlFardan and Kenneth G. Paterson. Lucky Thirteen: Breaking the TLS and DTLS Record Protocols (IEEE S&P 2013)

Introduction

The content itself may not be as "network security" as the others, but it relates to many real-world security issues.

1. Introduction

2. Information Flow
   - Noninterference
   - Non-deducibility
   - Covert Channels
   - Side-Channel Attacks

3. CBC Padding Oracle Attack
   - PKCS#7 Padding
   - Vaudenay's Attack
   - Side-Channel Attack
   - Constant-Time Cryptography

Information Flow

Information Flow

- A concept in Multi-level Security (MLS)
- In a program or system, we want to reason about how the "information" flows from varialbes to variables.
- Security labels
  - Simplest case: High and Low
  - High = Highly sensitive, Low = Low sensitivity
  - A program takes in inputs with low and high sensitivity, return things with low and high sensitivity
  - Assume: high inputs are secret, low inputs are public; high outputs are secret, low outputs are public
- If a user input (low) flows to commands in the server (high output), then bad things may happen.
  - E.g., SQL injection, if user input is used to construct a SQL query without sanitization

Example

A password checker takes in a user-input password (as low input), a secret password (as high input) and outputs whether the password is correct (as public, or low output).

Noninterference

- Model information flow with inputs and outputs
- The idea that the high inputs should not affect, or flow to, the low outputs.
- If a program fail to satisfy noninterference, then in a sense, the high inputs are leaked into the low outputs.
- If a program does satisfy noninterference, then given the output, one learns nothing about the high inputs.

Noninterference

- More formally, let P be a deterministic program, and I_H and I_L be the high and low inputs respectively, and O_H and O_L be the high and low outputs respectively, so $P(I_H, I_L) = (O_H, O_L)$.
- Then we say P has the **noninterference** property if for any I_{H_1} and I_{H_2} and any I_L, if we have
$$\begin{cases} P(I_{H_1}, I_L) = (O_{H_1}, O_{L_1}) \\ P(I_{H_2}, I_L) = (O_{H_2}, O_{L_2}) \end{cases}, \text{ then } O_{L_1} = O_{L_2}.$$
- We also say that there is no information flow between I_H and O_L.
- Notice that the property has to quantify over two traces of the program, so noninterference is also called a *hyperproperty*.

Example

A function f that takes one high and one low input, and returns one low output with $f(x, y) = x + y - x$.

Example

A function f that takes one high and one low input, and returns one low output with $f(x, y) = x + y - x$. Then it is easy to see that f has the noninterference property.

Example

A function f that takes one high and one low input, and returns one low output with $f(x, y) = x + y - x$. Then it is easy to see that f has the noninterference property.

Example (Encryption)

A function f that takes one high and two low inputs, and returns one low output with $f(m, n, e) = m^e \pmod{n}$.

Example

A function f that takes one high and one low input, and returns one low output with $f(x, y) = x + y - x$. Then it is easy to see that f has the noninterference property.

Example (Encryption)

A function f that takes one high and two low inputs, and returns one low output with $f(m, n, e) = m^e \pmod{n}$. f does not have non-interference, because the low input certainly depends on m.

Example

A function f that takes one high and one low input, and returns one low output with $f(x, y) = x + y - x$. Then it is easy to see that f has the noninterference property.

Example (Encryption)

A function f that takes one high and two low inputs, and returns one low output with $f(m, n, e) = m^e \pmod{n}$. f does not have non-interference, because the low input certainly depends on m.

Example (Password Checking)

Example

A function f that takes one high and one low input, and returns one low output with $f(x, y) = x + y - x$. Then it is easy to see that f has the noninterference property.

Example (Encryption)

A function f that takes one high and two low inputs, and returns one low output with $f(m, n, e) = m^e \pmod{n}$. f does not have non-interference, because the low input certainly depends on m.

Example (Password Checking)

The password checking function certainly does not have noninterference, because the output of the function depends on the high input (the secret password).

Example

A function f that takes one high and one low input, and returns one low output with $f(x, y) = x + y - x$. Then it is easy to see that f has the noninterference property.

Example (Encryption)

A function f that takes one high and two low inputs, and returns one
non

However, encryption and password checking are not really insecure.

Example (Password Checking)

The password checking function certainly does not have noninterference, because the output of the function depends on the high input (the secret password).

Problem with non-interference

- Essentially, noninterference is very strong
- Too strong to be realized in any real-world system
- Not easy to define for non-deterministic programs

Non-deducibility

- Similar to non-interference, but instead of learning nothing about the inputs from the outputs, we do not want to be able to eliminate (high) input from the set of possible input.
- Works for non-deterministic programs
- For any output O_L in the domain of P and for all I_H, there exists a low input I_L such that the low output of $P(I_H, I_L)$ is O_L.
- The negation would be that there exist some O_L such that we have I_H where no I_L can be used to produce O_L.
- This means that after observing the low output O_L, we can rule out the high input I_H.

Example (Password Checking)

The password checking function f can return True or False as low outputs. For every secret password, we have one low input such that f returns True, and many low inputs such that f returns False. Thus, the password checking function has the non-deducibility property.

Example (Password Checking)

The password checking function f can return True or False as low outputs. For every secret password, we have one low input such that f returns True, and many low inputs such that f returns False. Thus, the password checking function has the non-deducibility property.

Example (Information Leak)

Consider a function f that takes a file as high input and a user-chosen index i as low input. If i is out of bounds, then f returns a random character. Otherwise, f returns the character at index i of the file 99.9% of the time, and returns a random character 0.1% of the time. It is left as exercise to show that this has non-deducible.

Example (Password Checking)

The password checking function f can return True or False as low outputs. For every secret password, we have one low input such tha
Fal
non-deducibility property.

However, the second function is certainly not secure!

Example (Information Leak)

Consider a function f that takes a file as high input and a user-chosen index i as low input. If i is out of bounds, then f returns a random character. Otherwise, f returns the character at index i of the file 99.9% of the time, and returns a random character 0.1% of the time. It is left as exercise to show that this has non-deducible.

Covert Channels

- In the functional viewpoint, we care about the pure output of the program.
- However, low outputs should include everything that can be observed by the user.
- Consider the program

```
int f(high int x, low int y) {
    if (x > 0) {
        usleep(1000);
    }
    return y;
}
```

- We can deduce the value of x from the timing of the output!

Side-Channel Attacks

- Covert channels are a way to violate security policies by intentionally leaking high data
- Sometimes the violation is unintentional
- We call these violations **side-channel attacks**
- Side-channel attacks can be timing attacks, power analysis attacks, etc.

CBC Padding Oracle Attack

Introduction

Now we shift our focus to a real-world attack on CBC. There is an attack that allows an attacker to decrypt ciphertexts without knowing the key. Attempts to fix the attack has lead to a side-channel variant of the attack.

CBC Padding Oracle Attack

- The attack is based on AES with CBC mode of operation
- Recall that CBC mode is malleable (What does it mean?)
  - $C_i = E_k(P_i \oplus C_{i-1})$ (where C_0 is the IV)
  - $P_i = D_k(C_1) \oplus C_{i-1}$
- Since AES only works on blocks of 16 bytes, we need to pad the plaintext to a multiple of 16 bytes.
- This is typically done with PKCS#7 padding.

PKCS#7 Padding

- Consider a plaintext P of length n bytes. We want to add $p = 16 - (n \mod 16)$ bytes of padding.
- For PKCS#7, the value of each padding byte is p.

[1] \20 is a whitespace in ASCII.

[2] \05 has value 05 as a raw byte, and not the ASCII text "05", which has byte value 3035_{16}.

PKCS#7 Padding

- Consider a plaintext P of length n bytes. We want to add $p = 16 - (n \mod 16)$ bytes of padding.
- For PKCS#7, the value of each padding byte is p.
- For example, if $P =$ `Hello\20World`[1], then $n = 11$, and we need to pad 5 bytes. The padded plaintext is thus "`Hello\20World\05\05\05\05\05`"[2]
- If l is already a multiple of 16, we still need to pad 16 bytes of padding with value 16 (\10).

[1] \20 is a whitespace in ASCII.

[2] \05 has value 05 as a raw byte, and not the ASCII text "05", which has byte value 3035_{16}.

Padding Removal

- Notice that while every string can be padded, not every string with length being a multiple of 16 corresponds to a valid unpadded plaintext.

Padding Removal

- Notice that while every string can be padded, not every string with length being a multiple of 16 corresponds to a valid unpadded plaintext.
- The unpad function would have to return an error if the padding is invalid.

Padding Removal

- Notice that while every string can be padded, not every string with length being a multiple of 16 corresponds to a valid unpadded plaintext.
- The unpad function would have to return an error if the padding is invalid.
- Thus, after decrypting the ciphertext, it is natural to check for the padding, and return an error if the padding is invalid.

Padding Removal

- Notice that while every string can be padded, not every string with length being a multiple of 16 corresponds to a valid unpadded plaintext.
- The unpad function would have to return an error if the padding is invalid.
- Thus, after decrypting the ciphertext, it is natural to check for the padding, and return an error if the padding is invalid.
- However, this leads to an attack where we can decrypt the ciphertext without knowing the key.
- Attributed to Vaudenay in 2002.
- A kind of adaptive chosen-ciphertext attack.

Serge Vaudenay. Security flaws induced by CBC padding—applications to SSL, IPSEC, WTLS... (EUROCRYPT 2002)

Padding Oracle

- Abstractly, let us assume that there is a black-box function $\mathcal{O}$ that takes a ciphertext C and returns whether the padding is valid.
- We will call this function a **padding oracle**.
- In practice, we may submit a ciphertext to a web server. If the padding is valid, the server may proceed with the decrypted data. Otherwise, the server may return an error message.
- This produces a one-bit signal that tells us whether the padding is valid.

Vaudenay's Attack

- Assume we have a ciphertext $C = (IV, C)$, and we know that the padding is valid, and that the padding length is p.
- Then we know that the last p bytes of the plaintext all have value p.
- Now we can maul IV, so that the last p bytes of C has value $p + 1$.

Vaudenay's Attack

- Assume we have a ciphertext $C = (IV, C)$, and we know that the padding is valid, and that the padding length is p.
- Then we know that the last p bytes of the plaintext all have value p.
- Now we can maul IV, so that the last p bytes of C has value $p + 1$.
- We can send the modified ciphertext to the oracle, and check if the padding is valid.

Vaudenay's Attack

- Assume we have a ciphertext $C = (IV, C)$, and we know that the padding is valid, and that the padding length is p.
- Then we know that the last p bytes of the plaintext all have value p.
- Now we can maul IV, so that the last p bytes of C has value $p + 1$.
- We can send the modified ciphertext to the oracle, and check if the padding is valid.
- If yes, then we know that byte before the last p bytes of the plaintext has value $p + 1$

Vaudenay's Attack

- Assume we have a ciphertext $C = (IV, C)$, and we know that the padding is valid, and that the padding length is p.
- Then we know that the last p bytes of the plaintext all have value p.
- Now we can maul IV, so that the last p bytes of C has value $p + 1$.
- We can send the modified ciphertext to the oracle, and check if the padding is valid.
- If yes, then we know that byte before the last p bytes of the plaintext has value $p + 1$
- If the padding is invalid, then maul the byte before the last p bytes of the IV until the padding is valid.

Illustration

Assume that for the original plaintext, $P_{15} = IV_{15} \oplus D(C)_{15} \neq 01$.

$$
\begin{array}{cccccc}
 & IV_0 & IV_1 & IV_2 & \cdots & IV_{15} \\
\oplus & & & D(C) & & \\
\hline
 & ? & ? & ? & \cdots & ?
\end{array}
$$

Brute force on x ...

Most of the time we will get invalid padding.

$$\left.\begin{array}{cccccc} & & & & & x \\ & IV_0 & IV_1 & IV_2 & \cdots & IV_{15} \\ \oplus & & & D(C) & & \\ \hline & ? & ? & ? & \cdots & \text{junk} \end{array}\right\}\text{New } IV$$

Until...

However, when we DO get a valid padding, we have

$$\left.\begin{array}{cccccc} & & & & & x^* \\ & IV_0 & IV_1 & IV_2 & \cdots & IV_{15} \\ \oplus & & & D(C) & & \\ \hline & ? & ? & ? & \cdots & \backslash\text{x01} \end{array}\right\}\text{New } IV$$

Thus we get the following relation:

$$\begin{cases} & IV_{15} \oplus D(C)_{15} = P_{15} \\ x^* \oplus & IV_{15} \oplus D(C)_{15} = \backslash\text{x01} \end{cases}$$

xor the two equations to obtain

$$x^* = P_{15} \oplus \backslash\text{x01}$$

Now xor both sides with \x01 and rearrange to get

$$P_{15} = x^* \oplus \backslash\text{x01}$$

so we have successfully recovered the last byte of the plaintext!

Thus we get the following relation:

$$\begin{cases} & IV_{15} \oplus D(C)_{15} = P_{15} \\ x^* \oplus & IV_{15} \oplus D(C)_{15} = \backslash\text{x01} \end{cases}$$

xor the two equations to obtain

$$x^* = P_{15} \oplus \backslash\text{x01}$$

No

We can rinse and repeat to recover the entire plaintext block!

so we have successfully recovered the last byte of the plaintext!

Analysis

- We only need at most 256 queries to the oracle to recover one byte of the plaintext.
- We can repeat this for every byte of the plaintext block.
- We can recover the entire plaintext block P in at most $256 \times 16 = 4096$ queries.

Recovering the original padding length

- Recovering the original padding length is easy.
- Assume the original ciphertext (IV, C) has correct padding length p.

Recovering the original padding length

- Recovering the original padding length is easy.
- Assume the original ciphertext (IV, C) has correct padding length p.
- If we maul one of $IV[16 - p..15]$ by xor-ing some arbitrary value, then the padding will be invalid. (Why?)

Recovering the original padding length

- Recovering the original padding length is easy.
- Assume the original ciphertext (IV, C) has correct padding length p.
- If we maul one of $IV[16-p..15]$ by xor-ing some arbitrary value, then the padding will be invalid. (Why?)
- If we maul one of $IV[0..16-p-1]$, then since we are not touching the padding, then the padding is still valid.

Recovering the original padding length

- Recovering the original padding length is easy.
- Assume the original ciphertext (IV, C) has correct padding length p.
- If we maul one of $IV[16-p..15]$ by xor-ing some arbitrary value, then the padding will be invalid. (Why?)
- If we maul one of $IV[0..16-p-1]$, then since we are not touching the padding, then the padding is still valid.
- So we have a distinguisher that can tell whether the original padding length is p or not.

Corollary

This shows that AES-CBC with PKCS#7 padding will fail to have a kind of IND-CPA security where the adversary also has access to a padding oracle (in addition to the encryption oracle).

Exercise

Show that we can generalize this to recover a three-block ciphertext (IV, C_1, C_2).

Exercise (Encryption Oracle (hard))

Show that besides recovering ciphertexts, we can actually encrypt arbitrary plaintexts with the padding oracle without knowing the key. (Hint: if you know the plaintext corresponding to some ciphertexts, then you can maul that ciphertext to correspond to any plaintext you want.)

Remedy...?

- The key idea in Vaudenay's attack is the fact that we possess a padding oracle from decryption failure.

Remedy...?

- The key idea in Vaudenay's attack is the fact that we possess a padding oracle from decryption failure.
- The obvious fix is to not return any information about the padding.

Remedy...?

- The key idea in Vaudenay's attack is the fact that we possess a padding oracle from decryption failure.
- The obvious fix is to not return any information about the padding.
- In TLS, for example, the server encrypts the alert message, so an attacker cannot distinguish between an unpad failure and an error further down the line.

TLS

- Consider TLS 1.0 to 1.2, where if AES-CBC is chosen as the ciphersuite, then the record protocol will perform a MAC-Encode-Encrypt operation to encrypt the plaintexts.

TLS

- Consider TLS 1.0 to 1.2, where if AES-CBC is chosen as the ciphersuite, then the record protocol will perform a MAC-Encode-Encrypt operation to encrypt the plaintexts.
- The MAC is first computed, the plaintext is then padded and encrypted.

TLS

- Consider TLS 1.0 to 1.2, where if AES-CBC is chosen as the ciphersuite, then the record protocol will perform a MAC-Encode-Encrypt operation to encrypt the plaintexts.
- The MAC is first computed, the plaintext is then padded and encrypted.
- If the padding is invalid, then the server will return an alert message without computing MAC.

TLS

- Consider TLS 1.0 to 1.2, where if AES-CBC is chosen as the ciphersuite, then the record protocol will perform a MAC-Encode-Encrypt operation to encrypt the plaintexts.
- The MAC is first computed, the plaintext is then padded and encrypted.
- If the padding is invalid, then the server will return an alert message without computing MAC.
- If an attacker messed with the non-padding part, unpad would succeed, but MAC would still fail, returning another (encrypted) alert message.

TLS

- Consider TLS 1.0 to 1.2, where if AES-CBC is chosen as the ciphersuite, then the record protocol will perform a MAC-Encode-Encrypt operation to encrypt the plaintexts.
- The MAC is first computed, the plaintext is then padded and encrypted.
- If the padding is invalid, then the server will return an alert message without computing MAC.
- If an attacker messed with the non-padding part, unpad would succeed, but MAC would still fail, returning another (encrypted) alert message.
- A timing difference between valid and invalid padding!

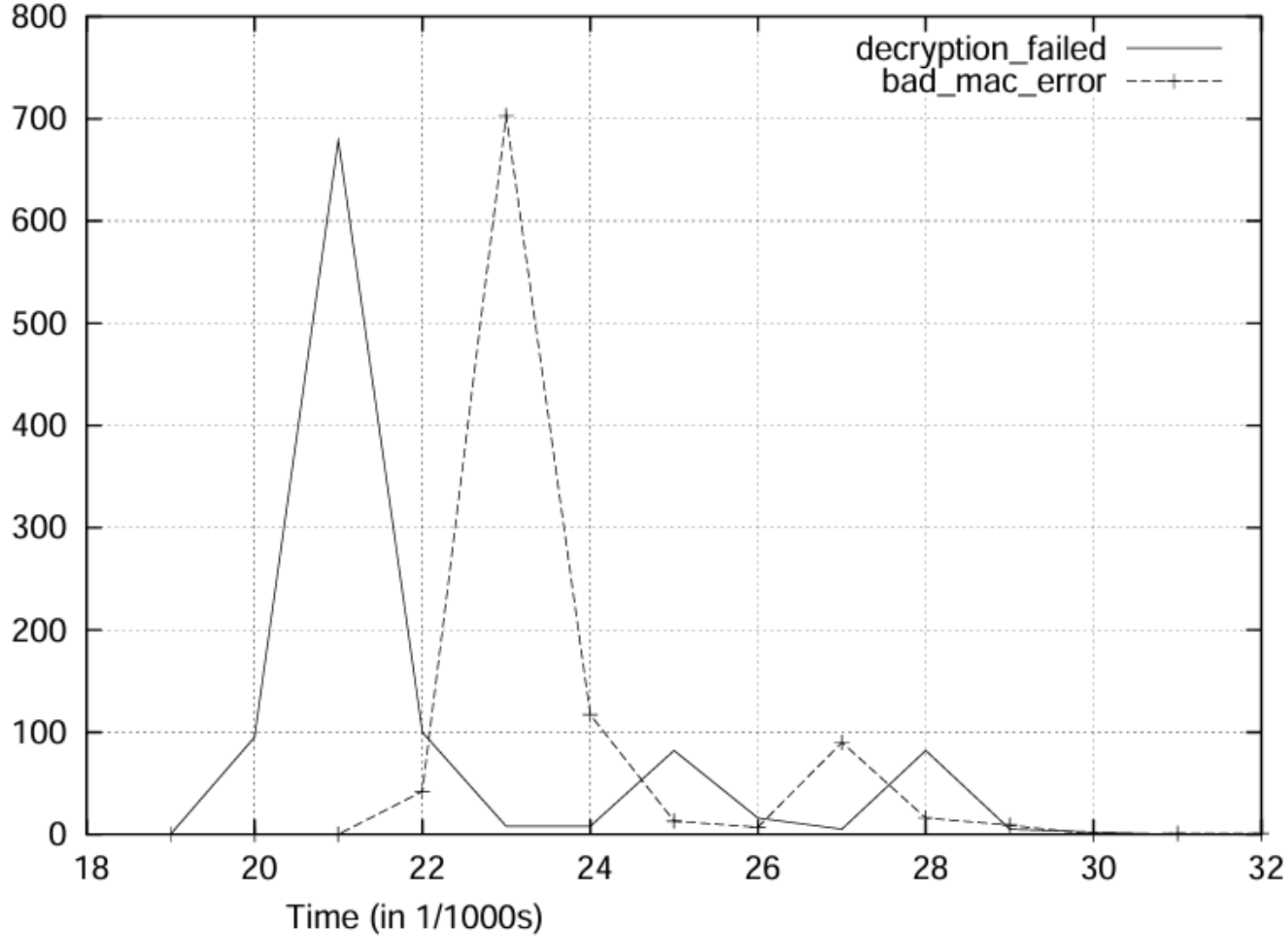
Brice Canvel, Alain Hiltgen, Serge Vaudenay, and Martin Vuagnoux. Password interception in a SSL/TLS channel. (CRYPTO 2003)

Side-Channel Attack on CBC

- Even though there is a timing difference, the difference is very small.
- In the network setting, there are lots of noise that affects the timing.
- We need to perform a lot of measurements, and perform standard statistical analysis to obtain a correct guess with high probability.
- Even if the attacker is physically close to the server, the random nature of noise means that the we need a lot of measurements to even deduce one bit of information, but this is possible

Side-Channel Attack on CBC

- Ev
sm
- In
tin
- W
sta
hig
- Ev
rar
me
is



Remedy...?

- We want to remove padding oracle

Remedy...?

- We want to remove padding oracle
- Make the unpad function constant-time w.r.t. input size

Remedy...?

- We want to remove padding oracle
- Make the unpad function constant-time w.r.t. input size
- Constant-time cryptography

Remedy...?

- We want to remove padding oracle
- Make the unpad function constant-time w.r.t. input size
- Constant-time cryptography
- Language-based security: Every pair of inputs to unpad (with the same padded length) should give the same "leakage"

Remedy...?

- We want to remove padding oracle
- Make the unpad function constant-time w.r.t. input size
- Constant-time cryptography
- Language-based security: Every pair of inputs to unpad (with the same padded length) should give the same "leakage"
- Practically, make the number of iterations of loops constant, eliminate branches that depend on (high/secret) input, use instructions that should be constant-time, ask the compiler not to optimize...

Remedy...? (Cont'd)

- If padding is valid but the MAC is not, then we get an alert
- If padding fails, we treat the decrypted junk as valid data to feed into the MAC (in other words, we treat padding=0 bytes)
- Then, MAC will still fail, and we will return an alert message.
- In either cases, MAC gets executed

Remedy...? (Cont'd)

RFC5246 (RFC for TLS1.2):
"This leaves a small timing channel, since MAC performance depends to some extent on the size of the data fragment, but it is not believed to be large enough to be exploitable, due to the large block size of existing MACs and the small size of the timing signal."

Remedy...? (Cont'd)

- If padding is valid but the MAC is not, then we get an alert
- If padding fails, we treat the decrypted junk as valid data to feed into the MAC (in other words, we treat padding=0 bytes)
- Then, MAC will still fail, and we will return an alert message.
- In either cases, MAC gets executed
- AlFardan and Paterson broke this approach in 2013.

Nadhem J. AlFardan and Kenneth G. Paterson. Lucky Thirteen: Breaking the TLS and DTLS Record Protocols (IEEE S&P 2013)

Remedy

- The real problem is that CBC is malleable.
- Encrypt-then-MAC instead of MAC-then-Encrypt?
- Instead, use authenticated encryption like AES-GCM.